

E-Commerce System Design with PostgreSQL

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

This document presents the full system design for a large-scale e-commerce platform, with PostgreSQL as the backbone. Rather than repeating the detailed case study from

`18_Architecture_Case_Studies/01_amazon_marketplace_db.md`, this document focuses on the architectural decisions: service boundaries, data flow, consistency requirements, and when each scaling technique becomes necessary. It covers catalog management, order processing, inventory, payments, and search — the five core domains of any e-commerce system.

Requirements

Functional Requirements

- Product catalog: browse, search, filter by price/rating/category
- Shopping cart: add/remove items, apply promo codes
- Checkout: place order, select shipping address and payment
- Inventory management: real-time stock tracking
- Order management: order history, status tracking, cancellation
- Payment processing: multiple methods, refunds
- Search: full-text, filters, autocomplete

Non-Functional Requirements

- 100M+ registered users
 - 1M+ orders per day
 - 500M+ product listings
 - P99 product page load < 200ms
 - P99 checkout < 500ms
 - Inventory consistency: strong (never oversell)
 - Payment idempotency: exactly-once semantics
 - 99.99% uptime for checkout
-

Capacity Estimation

Metric	Value
Daily active users	10M
Product page views per day	1B
Searches per day	200M
Add-to-cart events per day	50M
Orders per day	1M
Order items (avg 2.5/order)	2.5M
Payments per day	1M

Storage Estimates

Domain	Annual Growth	Notes
Product catalog	+100GB/yr	Slow-changing, highly cached
Orders	+300GB/yr	Write-once, append-mostly
Inventory	~50GB steady	Updated frequently
Search index	+200GB/yr	Elasticsearch / PG FTS
Click/analytics	+5TB/yr	Time-series, partitioned

QPS Breakdown

Operation	Peak QPS	DB Reads	DB Writes
Product page view	12K	Cache: 11.9K, DB: 100	0
Search	2.5K	Elasticsearch/PG	0
Add to cart	600	Redis: 600	0 (async)
Checkout	12	PostgreSQL	12
Order status	500	Read replica	0

Schema Design

```
-- =====
-- DOMAIN 1: CATALOG SERVICE DATABASE
-- =====
-- (Separate PostgreSQL instance or schema for the catalog domain)

CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
```

```

CREATE EXTENSION IF NOT EXISTS "pg_trgm";

CREATE TABLE categories (
  category_id SERIAL PRIMARY KEY,
  parent_id INT REFERENCES categories(category_id),
  name VARCHAR(200) NOT NULL,
  slug VARCHAR(200) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_category_slug UNIQUE (slug)
);

CREATE TABLE products (
  product_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  seller_id UUID NOT NULL,
  category_id INT NOT NULL REFERENCES categories(category_id),
  title VARCHAR(500) NOT NULL,
  slug VARCHAR(500) NOT NULL,
  description TEXT,
  brand VARCHAR(200),
  avg_rating NUMERIC(3,2),
  review_count INT NOT NULL DEFAULT 0,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  search_vector TSVECTOR,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE product_variants (
  variant_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  product_id UUID NOT NULL REFERENCES products(product_id),
  sku VARCHAR(200) NOT NULL,
  attributes JSONB NOT NULL DEFAULT '{}',
  price NUMERIC(12,2) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_sku UNIQUE (sku),
  CONSTRAINT chk_price CHECK (price > 0)
);

-- =====
-- DOMAIN 2: CART SERVICE (Redis-primary, PostgreSQL for persistence)
-- =====
-- Cart data lives in Redis: HASH key 'cart:{user_id}'
-- PostgreSQL stores persistent/saved carts only

CREATE TABLE saved_carts (
  cart_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL,
  items JSONB NOT NULL, -- [{variant_id, quantity,
price_snapshot}]
  promo_code VARCHAR(50),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

```

```

        CONSTRAINT uq_user_cart UNIQUE (user_id)
    );

-- =====
-- DOMAIN 3: INVENTORY SERVICE DATABASE
-- =====

CREATE TABLE inventory (
    variant_id          UUID          PRIMARY KEY,    -- same as
product_variants.variant_id
    warehouse_id        UUID          NOT NULL,
    quantity_on_hand     INT           NOT NULL DEFAULT 0,
    quantity_reserved     INT           NOT NULL DEFAULT 0,
    -- Computed: on_hand - reserved
    quantity_available   INT           GENERATED ALWAYS AS (quantity_on_hand -
quantity_reserved) STORED,
    updated_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_on_hand CHECK (quantity_on_hand >= 0),
    CONSTRAINT chk_reserved CHECK (quantity_reserved >= 0)
);

CREATE TABLE inventory_reservations (
    reservation_id       UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id             UUID          NOT NULL,
    variant_id           UUID          NOT NULL,
    quantity             INT           NOT NULL,
    reserved_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at           TIMESTAMPTZ NOT NULL,    -- released if order not confirmed
    released_at          TIMESTAMPTZ,
    status               VARCHAR(20) NOT NULL DEFAULT 'active',
    CONSTRAINT uq_order_variant UNIQUE (order_id, variant_id)
);

-- =====
-- DOMAIN 4: ORDER SERVICE DATABASE
-- =====

CREATE TYPE order_status AS ENUM (
    'pending_payment', 'payment_failed', 'confirmed',
    'processing', 'shipped', 'delivered', 'cancelled', 'refunded'
);

CREATE TABLE orders (
    order_id             UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_number         VARCHAR(30) NOT NULL,
    user_id              UUID          NOT NULL,
    status               order_status NOT NULL DEFAULT 'pending_payment',
    currency              CHAR(3)      NOT NULL DEFAULT 'USD',
    subtotal             NUMERIC(14,2) NOT NULL,
    discount_total       NUMERIC(14,2) NOT NULL DEFAULT 0,
    shipping_total       NUMERIC(14,2) NOT NULL DEFAULT 0,
    tax_total            NUMERIC(14,2) NOT NULL DEFAULT 0,
    grand_total          NUMERIC(14,2) NOT NULL,
    shipping_addr         JSONB        NOT NULL,    -- snapshot at order time

```

```

    placed_at      TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at     TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_order_number UNIQUE (order_number)
) PARTITION BY RANGE (placed_at);

CREATE TABLE order_items (
    item_id        UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id       UUID              NOT NULL REFERENCES orders(order_id),
    variant_id     UUID              NOT NULL,
    seller_id      UUID              NOT NULL,
    quantity       SMALLINT          NOT NULL,
    unit_price     NUMERIC(12,2)     NOT NULL,
    total_price    NUMERIC(12,2)     NOT NULL,
    CONSTRAINT chk_qty CHECK (quantity > 0)
);

-- =====
-- DOMAIN 5: PAYMENT SERVICE DATABASE
-- =====

CREATE TYPE payment_status AS ENUM (
    'pending', 'authorized', 'captured', 'failed', 'refunded'
);

CREATE TABLE payments (
    payment_id     UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id       UUID              NOT NULL,
    idempotency_key VARCHAR(100)     NOT NULL,
    amount         NUMERIC(14,2)     NOT NULL,
    currency       CHAR(3)           NOT NULL DEFAULT 'USD',
    status         payment_status    NOT NULL DEFAULT 'pending',
    provider       VARCHAR(50)       NOT NULL,
    provider_txn_id VARCHAR(200),
    created_at     TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at     TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_idempotency UNIQUE (idempotency_key),
    CONSTRAINT uq_order_payment UNIQUE (order_id)
);

-- =====
-- PROMO CODES (Shared Service)
-- =====

CREATE TABLE promo_codes (
    code           VARCHAR(50)       PRIMARY KEY,
    discount_type  VARCHAR(20)       NOT NULL, -- 'percent', 'fixed'
    discount_value NUMERIC(10,2)     NOT NULL,
    max_uses       INT,
    current_uses   INT               NOT NULL DEFAULT 0,
    min_order_value NUMERIC(10,2),
    valid_from     TIMESTAMPTZ      NOT NULL,
    valid_until    TIMESTAMPTZ,
    is_active      BOOLEAN           NOT NULL DEFAULT TRUE
);

```

```

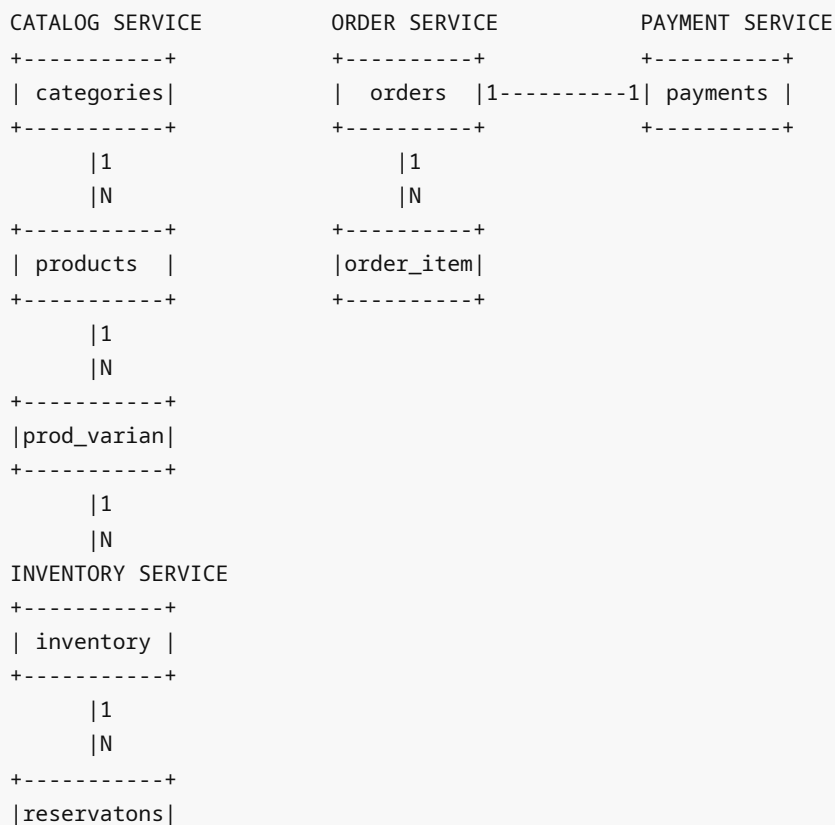
CREATE TABLE promo_usage (
  code          VARCHAR(50)      NOT NULL REFERENCES promo_codes(code),
  user_id       UUID             NOT NULL,
  order_id      UUID             NOT NULL,
  discount_applied NUMERIC(10,2) NOT NULL,
  used_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
  PRIMARY KEY (code, user_id, order_id)
);

-- =====
-- SEARCH (PostgreSQL FTS or Elasticsearch sync table)
-- =====
-- If using PostgreSQL FTS (up to ~50M products):
CREATE INDEX idx_products_fts ON products USING GIN (search_vector);

-- Trigger to maintain search vector
CREATE TRIGGER products_search_update
  BEFORE INSERT OR UPDATE OF title, description, brand
  ON products
  FOR EACH ROW EXECUTE FUNCTION tsvector_update_trigger(
    search_vector, 'pg_catalog.english', title, description
  );

```

ASCII ER Diagram



```

+-----+
CART (Redis)                                PROMO SERVICE
[user:{id}] → [{variant,qty}]               +-----+
                                           |promo_codes|
                                           +-----+
                                           | 1
                                           |N
                                           +-----+
                                           |promo_usage|
                                           +-----+

```

Indexing Strategy

```

-- CATALOG SERVICE
CREATE INDEX idx_products_category      ON products (category_id, is_active);
CREATE INDEX idx_products_seller        ON products (seller_id, created_at DESC);
CREATE INDEX idx_products_rating        ON products (avg_rating DESC NULLS LAST)
WHERE is_active;
CREATE INDEX idx_variants_product       ON product_variants (product_id) WHERE
is_active;
CREATE INDEX idx_variants_sku           ON product_variants (sku);

-- INVENTORY SERVICE (hot path – called at checkout and product display)
CREATE INDEX idx_inventory_available     ON inventory (variant_id)
WHERE quantity_available > 0;
CREATE INDEX idx_reservations_order     ON inventory_reservations (order_id);
CREATE INDEX idx_reservations_expiry    ON inventory_reservations (expires_at,
status)
WHERE status = 'active';

-- ORDER SERVICE
CREATE INDEX idx_orders_user            ON orders (user_id, placed_at DESC);
CREATE INDEX idx_orders_status          ON orders (status, placed_at)
WHERE status NOT IN ('delivered', 'cancelled', 'refunded');
CREATE INDEX idx_order_items_order      ON order_items (order_id);
CREATE INDEX idx_order_items_seller     ON order_items (seller_id, order_id);

-- PAYMENT SERVICE
CREATE INDEX idx_payments_order         ON payments (order_id);
CREATE INDEX idx_payments_pending       ON payments (status, created_at)
WHERE status IN ('pending', 'authorized');

```

Query Patterns

Q1 — Product Detail Page with Availability

```

SELECT
    p.product_id, p.title, p.description, p.avg_rating, p.review_count,
    json_agg(DISTINCT jsonb_build_object(
        'variant_id', pv.variant_id,
        'sku', pv.sku,
        'price', pv.price,
        'attributes', pv.attributes,
        'available', i.quantity_available > 0
    )) AS variants
FROM products p
JOIN product_variants pv ON pv.product_id = p.product_id AND pv.is_active
LEFT JOIN inventory i ON i.variant_id = pv.variant_id
WHERE p.product_id = $1 AND p.is_active
GROUP BY p.product_id;

```

Q2 — Checkout: Reserve Inventory + Create Order (Saga Step 1)

```

-- Step 1: Reserve inventory (per variant, atomic)
BEGIN;

-- Attempt reservation
UPDATE inventory
SET quantity_reserved = quantity_reserved + $qty
WHERE variant_id = $variant_id
    AND quantity_available >= $qty;

GET DIAGNOSTICS v_rows_updated = ROW_COUNT;
IF v_rows_updated = 0 THEN
    ROLLBACK;
    RAISE EXCEPTION 'out_of_stock';
END IF;

INSERT INTO inventory_reservations
    (reservation_id, order_id, variant_id, quantity, expires_at)
VALUES
    (uuid_generate_v4(), $order_id, $variant_id, $qty, now() + INTERVAL '15
minutes');

COMMIT;

```

Q3 — Create Order (after payment authorized)

```

BEGIN;

INSERT INTO orders (order_id, order_number, user_id, status, subtotal, grand_total,
shipping_addr)
VALUES ($order_id, $order_number, $user_id, 'confirmed', $subtotal, $grand_total,
$addr_json);

```



```

INSERT INTO order_items (order_id, variant_id, seller_id, quantity, unit_price,
total_price)
VALUES ($order_id, $variant_id, $seller_id, $qty, $price, $total);

-- Convert reservations to actual deductions
UPDATE inventory
SET quantity_reserved = quantity_reserved - $qty,
    quantity_on_hand = quantity_on_hand - $qty,
    updated_at = now()
WHERE variant_id = $variant_id;

UPDATE inventory_reservations
SET status = 'fulfilled', released_at = now()
WHERE order_id = $order_id AND variant_id = $variant_id;

COMMIT;

```

Q4 — Search Products (PostgreSQL FTS)

```

SELECT
    p.product_id, p.title, p.avg_rating,
    MIN(pv.price) AS min_price,
    ts_rank(p.search_vector, query) AS rank
FROM products p,
    to_tsquery('english', $1) query
JOIN product_variants pv ON pv.product_id = p.product_id AND pv.is_active
WHERE p.search_vector @@ query AND p.is_active
ORDER BY rank DESC, p.avg_rating DESC NULLS LAST
LIMIT 50;

```

Q5 — Category Browse with Price Filter

```

SELECT
    p.product_id, p.title, p.avg_rating,
    MIN(pv.price) AS min_price,
    MAX(pv.price) AS max_price
FROM products p
JOIN product_variants pv ON pv.product_id = p.product_id AND pv.is_active
WHERE p.category_id = $category_id
    AND p.is_active
    AND pv.price BETWEEN $min_price AND $max_price
GROUP BY p.product_id
ORDER BY p.avg_rating DESC NULLS LAST
LIMIT 50 OFFSET $offset;

```

Q6 — Order History

```

SELECT
    o.order_id, o.order_number, o.status,
    o.grand_total, o.placed_at,
    json_agg(jsonb_build_object(
        'variant_id', oi.variant_id,
        'quantity', oi.quantity,
        'price', oi.total_price
    )) AS items
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.user_id = $1
ORDER BY o.placed_at DESC
LIMIT 20 OFFSET $2;

```

Q7 — Cancel Order (release inventory)

```

BEGIN;

UPDATE orders
SET status = 'cancelled', updated_at = now()
WHERE order_id = $order_id AND status IN ('confirmed', 'processing');

-- Release inventory
UPDATE inventory i
SET quantity_reserved = quantity_reserved - oi.quantity,
    quantity_on_hand = quantity_on_hand + oi.quantity,
    updated_at = now()
FROM order_items oi
WHERE oi.order_id = $order_id AND oi.variant_id = i.variant_id;

COMMIT;

```

Q8 — Apply Promo Code

```

BEGIN;

-- Validate and claim the promo slot
UPDATE promo_codes
SET current_uses = current_uses + 1
WHERE code = $code
    AND is_active
    AND (max_uses IS NULL OR current_uses < max_uses)
    AND valid_from <= now()
    AND (valid_until IS NULL OR valid_until >= now())
RETURNING discount_type, discount_value;

-- Record usage
INSERT INTO promo_usage (code, user_id, order_id, discount_applied)
VALUES ($code, $user_id, $order_id, $discount);

```

```
COMMIT;
```

Q9 — Expiry Stale Reservations (scheduled job)

```
BEGIN;

-- Release stale reservations
UPDATE inventory i
SET quantity_reserved = quantity_reserved - r.quantity,
    updated_at         = now()
FROM inventory_reservations r
WHERE r.variant_id = i.variant_id
    AND r.status = 'active'
    AND r.expires_at < now();

UPDATE inventory_reservations
SET status = 'expired', released_at = now()
WHERE status = 'active' AND expires_at < now();

COMMIT;
```

Q10 — Seller Sales Report

```
SELECT
    DATE_TRUNC('day', o.placed_at) AS day,
    COUNT(DISTINCT o.order_id) AS orders,
    SUM(oi.quantity) AS units_sold,
    SUM(oi.total_price) AS gross_revenue
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
WHERE oi.seller_id = $1
    AND o.status IN ('confirmed', 'processing', 'shipped', 'delivered')
    AND o.placed_at >= NOW() - INTERVAL '30 days'
GROUP BY 1
ORDER BY 1 DESC;
```

Scaling Strategy

Service Boundary Evolution

Phase 1 — Monolith (0–1M users) Single PostgreSQL database. All domains in one schema. Simple and fast to develop.

Phase 2 — Domain Databases (1M–50M users) Split into dedicated databases per service:

- CatalogDB: products, categories, variants
- InventoryDB: stock levels, reservations
- OrderDB: orders, order_items

- PaymentDB: payments, refunds
- SearchDB (Elasticsearch): product search

Each service communicates via API; no cross-service joins.

Phase 3 — Horizontal Scaling (50M+ users)

Catalog: Mostly read-only → aggressive caching (Redis, CDN). PostgreSQL primary + 3 read replicas.

Inventory: High contention → Redis for fast reservation checks; PostgreSQL for reconciliation and permanent deduction.

Orders: Partition by `placed_at` quarterly. Orders older than 2 years archived to cold storage.

Search: Elasticsearch cluster, synced from PostgreSQL via Debezium (CDC). PostgreSQL is the source of truth; Elasticsearch serves all search queries.

The Checkout Saga Pattern

1. CartService: Validate cart items and prices
2. PromoService: Validate and reserve promo code
3. InventoryService: Reserve stock (PostgreSQL transaction per item)
4. PaymentService: Authorize payment (`idempotency_key = order_id`)
5. OrderService: Create order record (confirmed status)
6. InventoryService: Fulfill reservations (deduct from on-hand)
7. NotificationService: Send confirmation email

Compensation (on failure at any step):

- Step 3 fail: rollback promo reservation
- Step 4 fail: release inventory reservations
- Step 5 fail: void payment authorization

Tradeoffs

Decision	Chosen	Alternative	Why
Saga for checkout	Distributed saga	2PC across services	2PC locks all services; saga allows independent scaling and failure recovery
Inventory reservation with expiry	15-minute reservation	Lock until payment	Lock could block other buyers indefinitely on payment failure; expiry auto-releases
Cart in Redis	Redis only	PostgreSQL	Cart is temporary, frequently updated (every add/remove); Redis is ideal for ephemeral state
PostgreSQL FTS vs. Elasticsearch	ES for production	PG FTS	ES handles relevance tuning, faceted search, multilingual better; PG FTS acceptable up to ~50M products

JSONB shipping address snapshot	Snapshot in order	FK to addresses	Buyer may update/delete address after order; snapshot preserves what was actually ordered
---------------------------------	-------------------	-----------------	---

Interview Discussion Points

- How does inventory reservation prevent overselling?** The inventory row's `quantity_available` is a generated column (`on_hand - reserved`). The UPDATE only succeeds if `quantity_available >= qty` in the WHERE clause — this is a single atomic SQL statement. If two concurrent checkouts target the same item, one will update 0 rows and retry.
- How do you handle the "cart price divergence" problem?** When a user adds an item to their cart at \$100, and the seller changes the price to \$90 before checkout, what should the user pay? Best practice: show current price at checkout (recalculate), alert user if it changed, allow them to proceed at new price. Store `price_snapshot` in cart for display, but recalculate at checkout time.
- How do you implement flash sales (Black Friday)?** Pre-generate a fixed number of inventory reservation tokens in Redis (`DECR inventory:{variant_id}:available`). Only buyers who successfully DECR to `>= 0` proceed to PostgreSQL checkout. PostgreSQL never sees the thundering herd.
- Why partition the orders table by date?** 99% of queries filter by date range (recent orders). Partitioning means PostgreSQL scans only the relevant partitions. Old partitions can be archived or dropped. The alternative — a single 100GB+ orders table — would require full table scans for most queries.

Common Interview Follow-ups

Q: How do you handle a payment that succeeds but the order creation fails? A: The payment has an `idempotency_key = order_id` . If order creation fails, the payment is in `authorized` state (not captured). A background reconciliation job detects `authorized` payments with no corresponding `confirmed` order older than 30 minutes and voids them. The payment provider is called to cancel the authorization.

Q: How do you scale the product catalog to 1B SKUs? A: Partition `products` by `category_id` range (each major category is a partition). Use Elasticsearch as the primary search and browse interface (not PostgreSQL). PostgreSQL stores the authoritative catalog data but is not the query engine for browse/search at that scale.

Q: How do you recommend products (recommendations engine)? A: Collaborative filtering: find users with similar purchase history (matrix factorization, run offline). Store results in a `recommendations(user_id, product_id, score)` table. Content-based: find products with similar attributes (TF-IDF on description, run offline). Both are pre-computed and stored in PostgreSQL/Redis; no ML at query time.

Performance Considerations

- Product page caching:** Cache the full product page response at the CDN/API gateway level (TTL 60s). For 1B page views/day, 99.9% cache hit rate means PostgreSQL handles only 1M requests/day.
- Inventory hot rows:** Flash sales concentrate writes on a few inventory rows. Use advisory locks or queue-based serialization to prevent lock contention on hot SKUs.

- **Order partitions and VACUUM:** High INSERT rate on recent partitions means frequent dead tuples from UPDATE (status changes). Set `autovacuum_vacuum_cost_delay = 2` for the orders table to keep VACUUM aggressive.
 - **Connection routing:** Use a proxy (ProxySQL, PgBouncer, Pgpool) to route read traffic to replicas. Tag queries with comments (`/* replica_ok */`) to help routing proxies.
-

Cross-References

- See `18_Architecture_Case_Studies/01_amazon_marketplace_db.md` for full schema details
- See `03_banking_system_design.md` for payment processing architecture
- See `07_analytics_platform_design.md` for order analytics pipeline
- See `08_system_design_framework.md` for interview presentation framework
- Partitioning: Chapter 5 of this guide
- Transactions and ACID: Chapter 4 of this guide